

1. Środowisko i narzędzia

- **System operacyjny:** Linux / Windows 11 z WSL2 (Ubuntu).
- **Język:** C++ (Standard C++11).
- **Kompilator:** GCC (g++).
- **Zarządzanie kompilacją:** Makefile.
- **Edytor:** CLion

2. Budowanie, uruchomienie i logi

Projekt wykorzystuje narzędzie **make** do automatyzacji procesu kompilacji.

Kompilacja:

- W katalogu głównym projektu należy wywołać komendę: **make**
Spowoduje to utworzenie plików wykonywalnych: **main**, **klient**, **kasjer**, **przewodnik**, **straznik** oraz **cleanup**.
- **Uruchomienie symulacji:**
System wymaga dwóch terminali do pełnej obsługi:
 - **Terminal 1 (Symulacja):**
./main – uruchamia zarządcę, który powołuje procesy potomne (turystów, kasjera, przewodnika) i inicjuje środowisko.
 - **Terminal 2 (Sterowanie):**
./straznik – uruchamia panel sterowania do wysyłania sygnałów (zamykanie tras).
- **Obsługa logów:**
Symulacja nie zaśmieca terminala logami operacyjnymi. Są one przekierowywane do plików w tle:
 - **raport_ruch.txt:** Szczegóły wejść/wyjść z kładki i tras.
 - **raport_kasjer.txt:** Historia sprzedaży biletów.
 - **Podgląd na żywo:**
Aby widzieć logi w czasie rzeczywistym, należy w osobnym terminalu wpisać:
tail -f raport_ruch.txt
- **Zakończenie symulacji:**
Wykonuje się automatycznie po zakończeniu generowaniu procesów. Automatycznie usuwa wszystkie zasoby IPC
Można symulację także zakończyć w alternatywny sposób:
 - **Ctrl+C** w Terminalu 1, aby posprzątać procesy.
 - Awaryjne usuwanie zasobów IPC: **./cleanup**

3. Opis projektu

Projekt jest symulacją współbieżną obsługi ruchu turystycznego w jaskini, zrealizowaną w modelu wielopprocesowym bez centralizacji. Głównym celem jest synchronizacja dostępu do zasobów współdzielonych (wąskie kładki, limitowane trasy) oraz obsługa priorytetów i zróżnicowanych typów klientów. Procesy (Kasjer, Turysta, Przewodnik) komunikują się wyłącznie poprzez mechanizmy IPC Systemu V (kolejki komunikatów, pamięć dzielona, semafor), co odwzorowuje rzeczywiste, niezależne zachowanie aktorów systemu.

4. Zrealizowane wymagania

Projekt spełnia kluczowe wymagania funkcjonalne narzucone przez temat:

- **Ruch wahadłowy na kładce:** Kładka o pojemności K (limit 3 osoby) obsługuje ruch tylko w jedną stronę w danej chwili (mechanizm w klient.cpp).
- **Dwie trasy z limitami:** Trasy o pojemnościach N1 i N2 (limity 10 i 15 osób) chronione semaforami.
- **Losowe przybycia:** Generator tworzy turystów w losowych odstępach czasu (usleep).
- **Symulacja czasu zwiedzania:** Czas T1 i T2 symulowany przez funkcję usleep wewnątrz procesu turysty.
- **System biletowy:** Zakup biletów w kasie z wykorzystaniem kolejki komunikatów.
- **Zniżki dla dzieci:** Dzieci < 3 lat otrzymują bilet darmowy (zliczane w statystykach kasjera).
- **Restrykcje wiekowe (Seniorzy/Dzieci):**
 - Dzieci < 8 lat (z opiekunem) kierowane wyłącznie na Trasę 2.
 - Seniorzy > 75 lat kierowani wyłącznie na Trasę 2.
- **Obsługa VIP (Powracający):** ~10% turystów to klienci powracający (zniżka 50% i priorytetowa obsługa w kolejce msgrcv).
- **Sterowanie sygnałami:** Strażnik wysyła sygnały SIGUSR1 (Trasa 1) i SIGUSR2 (Trasa 2) do Przewodnika.
- **Dynamiczne zamykanie tras:** Po otrzymaniu sygnału, nowe grupy nie są wpuszczane, a oczekujący rezygnują z wejścia.
- **Raportowanie:** Generowanie plików tekstowych z przebiegu symulacji.

5. Struktura kodu

main.cpp (Manager)

- Inicjalizuje zasoby IPC: semafony (mutex, limity tras/mostu), pamięć dzieloną (**CaveState**) oraz kolejkę komunikatów.
- Ustawia wartości początkowe semaforów (limity miejsc, statusy otwarcia).
- Uruchamia procesy potomne: **kasjer**, **przewodnik** oraz **klient** (generator) przy użyciu **fork()** i **execlp()**.
- Przekierowuje wyjście standardowe (**stdout**) procesów potomnych do plików logów (**dup2**).
- Obsługuje sygnał **SIGINT** (Ctrl+C), zapewniając zabicie procesów potomnych i usunięcie zasobów IPC (**cleanup_resources**).

klient.cpp (Generator i Turysta)

- Generuje w pętli procesy potomne symulujące pojedynczych turystów.
- Implementuje logikę turysty: losuje wiek, decyduje o byciu "powracającym" (VIP), dobiera trasę do wieku.
- Wysyła żądanie biletu do Kasjera i czeka na obsługę.
- Realizuje algorytm wejścia na most: sprawdza w sekcji krytycznej kierunek ruchu i liczbę osób (**bridge_direction**, **people_on_bridge**).
- Reaguje na zamknięcie trasy: jeśli flaga w pamięci dzielonej to 0, turysta rezygnuje i zwalnia zasoby.

kasjer.cpp (System Biletowy)

- Działa w pętli nieskończonej, nasłuchując na kolejce komunikatów.
- Wykorzystuje priorytetowe odbieranie wiadomości (**msgrcv** z typem **-2**), obsługując VIP-ów (**mtype=1**) przed zwykłymi klientami (**mtype=2**).
- Aktualizuje statystyki finansowe w pamięci dzielonej (bilety sprzedane / darmowe).
- Loguje transakcje do pliku raportu.

przewodnik.cpp (Monitoring)

- Podłącza się do pamięci dzielonej i cyklicznie odczytuje stan jaskini (liczniki osób, stan mostu).
- Wizualizuje status systemu w terminalu (tabela statusów).
- Rejestruje obsługę sygnałów **SIGUSR1** i **SIGUSR2**.
- W reakcji na sygnał zmienia flagi **route1_open** / **route2_open** w pamięci dzielonej (blokada wejścia).

straznik.cpp (Sterowanie)

- Pobiera PID procesów **przewodnik i manager** z pamięci dzielonej.
- Umożliwia użytkownikowi wysłanie sygnałów systemowych (**kill**) w celu zamknięcia tras lub zakończenia symulacji.

tools.cpp

- Zawiera wrappery na funkcje systemowe IPC (**semop**, **shmget**, **msgsnd**), implementując w nich obsługę błędów (**perror**).
- Udostępnia funkcje pomocnicze do logowania (**safe_log**) i synchronizacji semaforów (**lock_sem**, **unlock_sem**).

6. Testy

Test 1: Weryfikacja przepustowości kładki (Limit K)

- **Opis:** Symulacja sytuacji dużego obciążenia w celu sprawdzenia, czy system poprawnie blokuje wejście na kładkę po osiągnięciu limitu $K=3$ osób, zmuszając nadmiarowe procesy do oczekiwania przed wejściem.
- **Oczekiwany rezultat:** Na kładce znajduje się maksymalnie K osób, reszta czeka w kolejce na zwolnienie semafora.
- **Kod realizujący:**
 - Plik **klient.cpp**: Warunek `if (jaskinia->people_on_bridge < LIMIT_BRIDGE)`

Test 2: Blokada ruchu dwukierunkowego (Wąskie gardło)

- **Opis:** Symulacja konfliktu grupy wchodzącej i wychodzącej w celu weryfikacji poprawności działania semaforów i flagi kierunku (**bridge_direction**), co ma zapobiec zakleszczeniom na moście.
- **Oczekiwany rezultat:** Grupa wchodząca musi zaczekać, aż kładka zostanie całkowicie opuszczona przez grupę wychodzącą (wyzerowanie flagi kierunku).
- **Kod realizujący:**
 - Plik **klient.cpp**: Warunek w `wejdz_na_most` `if (jaskinia->bridge_direction == 0 ...)` oraz w `zejdz_z_mostu` `if (jaskinia->people_on_bridge == 0) ...` weryfikujący zgodność kierunku.

Test 3: Weryfikacja reguł biletowych i wieku

- **Opis:** Próba zakupu biletu przez osobę w wieku 80 lat na Trasę nr 1 oraz dziecka bez opiekuna, w celu sprawdzenia czy system automatycznie egzekwuje regulamin przydziału tras.
- **Oczekiwany rezultat:** System automatycznie przydziela Trasę 2 dla seniorów i dzieci, uniemożliwiając wybór Trasy 1.

- **Kod realizujący:**
 - Plik **klient.cpp** Instrukcja warunkowa [if \(age > AGE_SENIOR\)](#) nadpisująca wybór trasy.

Test 4: Obsługa sygnału od Strażnika

- **Opis:** Wysłanie sygnału systemowego przez proces Strażnika do Przewodnika podczas oczekiwania grupy na wejście, w celu weryfikacji dynamicznego zamykania trasy.
- **Oczekiwany rezultat:** Przewodnik zmienia status trasy na ZAMKNIĘTĄ, a nowi turyści rezygnują z wejścia i kończą proces.
- **Kod realizujący:**
 - Plik **przewodnik.cpp**: Handler sygnału [przelacz_trase1](#)
 - Plik **klient.cpp**: Reakcja turysty na zamkniętą trasę [exit](#)

7. Funkcje wymagane przez projekt

a. Tworzenie i obsługa plików

- [open\(\)](#)
- [dup2\(\)](#)
- [close\(\)](#)

b. Tworzenie procesów

- [fork\(\)](#)
- [execvp\(\)](#)
- [wait\(\)](#)
- [exit\(\)](#)

d. Obsługa sygnałów

- [signal\(\)](#)
- [kill\(\)](#)

e. Synchronizacja procesów

- [semop\(\)](#)
- [semget\(\)](#)

g. Segmenty pamięci dzielonej

- [shmget\(\)](#)
- [shmat\(\)](#)

h. Kolejki komunikatów

- [msgsnd\(\)](#)
- [msgrcv\(\)](#)